Анализ возможностей современных инструментов разработки языков программирования

Оглавление

1. Введение
    1. Актуальность темы
    2. Цель и задачи работы
    3. Объект и предмет исследования
    4. Методология и подходы
2. Теоретическая часть
    1. Анализ существующих языков программирования и их особенностей
        1. Классификация языков программирования
        2. Популярные языки и их особенности
        3. Недостатки языков
        4. Выводы по результатам анализа
        5. Формулировка требований функционала к инструментам разработки языков
    2. Архитектура интерпретатора
        1. Общая схема компилятора (лексический, синтаксический анализ, генерация кода)
        2. Лексический анализ (токенизация, регулярные выражения,)
        3. Синтаксический анализ (контекстно-свободные грамматики, деревья разбора)
        4. Семантический анализ (проверка типов, области видимости, таблицы символов)
        5. Интерпретация
3. Практическая часть
    1. Реализация интерпретатора
        1. Выбор инструментов и технологий (язык реализации, библиотеки)
        2. Структура программы интерпретатора
        3. Описание языка программирования Aqua
            1. Синтаксис и семантика языка
            2. Типы данных и конструкции управления
            3. Особенности и отличия от других языков
            4. Примеры программ на Aqua
    2. Реализация лексического анализатора
    3. Реализация парсера

    4. Реализация синтаксического анализатора
    5. Реализация семантического анализатора
    6. Выполнение кода
    7. Тесты
4. Заключение
    2. Оценка производительности языка
    3. Достигнутые цели
    4. Перспективы развития
    5. Выводы
5. Список использованных источников
6. Приложения

Введение

Актуальность темы

В современных условиях стремительного развития информационных технологий языки программирования играют ключевую роль в создании программного обеспечения различной сложности.
Популярные языки программирования предлагают различные подходы и инструменты для написания кода, однако зачастую они имеют избыточный синтаксис, высокую сложность освоения, наследуют структурные недостатки предшественников и недостаточную выразительность в определённых предметных областях. Это затрудняет освоение начинающими разработчиками, выпуск новых квалифицированных кадров, усложняет поиск ошибок, увеличивает время разработки и отладки, и создаёт трудности при поддержке приложения.
Постоянное развитие программных систем, рост требований к производительности, безопасности и масштабируемости приводят к необходимости разработки новых решений.

Разработка нового языка программирования позволяет применить совершенно новые подходы в семантическом и синтаксическом анализе для решения практических задач. Таким образом, анализ современных инструментов разработки языков программирования нацелен на изучение внедрения новых подходов для дальнейшей разработки языка программирования, обладающим такими преимуществами, как: простота использования, хорошая читаемость, широкая функциональность и достаточная выразительность с поддержкой современных подходов программирования и автоматическим поиском ошибок на этапе анализа.

Особую значимость приобретают инструменты разработки языков программирования, такие как генераторы лексических и синтаксических анализаторов, системы построения компиляторов и интерпретаторов, а также средства автоматизации анализа кода. Эти инструменты позволяют существенно сократить время разработки языков, повысить их надёжность и упростить сопровождение.

Актуальность данной работы обусловлена необходимостью анализа возможностей современных инструментов разработки языков программирования, а также изучения подходов к созданию интерпретаторов как одного из наиболее доступных способов реализации языков.

Цель и задачи работы

Целью данной работы является анализ возможностей современных инструментов разработки языков программирования и практическая реализация интерпретатора для учебного языка Aqua.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ существующих языков программирования и их особенностей;
- изучить архитектуру, использующуюся в реализации компиляторов и интерпретаторов;
- исследовать современные инструменты разработки языков (лексеры, парсеры и др.);
- определить подходящие инструменты для реализации языка;
- спроектировать структуру интерпретатора;
- реализовать основные компоненты интерпретатора (лексический анализатор, парсер, семантический анализатор, исполнитель);
- разработать и описать язык программирования Aqua;
- провести тестирование и оценку работы интерпретатора.
- определить последующие шаги развития

Объект и предмет исследования

Объектом исследования являются инструменты разработки языков программирования, а также процессы создания интерпретаторов.

Предметом исследования выступают методы и средства реализации интерпретаторов, включая:

- Теория формальных языков, методы лексического, синтаксического и семантического анализа;
- применение генераторов анализаторов, таких как jflex и byacc/j и вспомогательных инструментов;
- проектирование структуры проекта для реализации интерпретатора;
- особенности реализации выполнения программ на разработанном языке Aqua.

Методология и подходы

В рамках данной работы используются следующие методы и подходы:

- **Аналитический метод** — для изучения существующих языков программирования и инструментов их разработки;
- **Сравнительный анализ** — для выявления преимуществ и недостатков различных подходов и технологий;
- **Формальные методы** — при описании синтаксиса языка с использованием контекстно-свободных грамматик;
- **Методы проектирования программного обеспечения** — при разработке архитектуры интерпретатора;
- **Экспериментальный метод** — при реализации и тестировании разработанного интерпретатора.

В качестве практического подхода используется поэтапная разработка интерпретатора, включающая реализацию лексического, синтаксического и семантического анализа с последующей интерпретацией программы. Такой подход позволяет наглядно продемонстрировать полный цикл обработки исходного кода и оценить возможности современных инструментов разработки языков программирования.

Теоретическая часть

Языки программирования являются основным инструментов взаимодействия программиста с компьютером. За время развития отрасли появилось множество языков, отличающихся синтаксисом, семантикой, областью применения, способом исполнения кода, скоростью работы, принципами работы с памятью и многими другими аспектами, поэтому необходимо рассмотреть их преимущества, недостатки, а также пробелы в функциональности.

Анализ существующих языков программирования и их особенностей

Разработка нового языка программирования требует предварительного анализа существующих решений, их преимуществ и недостатков. Такой анализ позволяет выявить ключевые проблемы современных языков и сформулировать требования к инструментам разработки компиляторов и интерпретаторов для устранения недостатков.

Одной из важных задач является исследование возможности выполнения статического анализа программы до её запуска. В частности, представляет интерес выполнение следующих проверок:

- автоматическое определение типов
- анализ достижимости кода и полноты ветвлений
- возможность реализации различных видов сокращённых конструкций, например, для цикла `for`

Реализация подобных механизмов позволяет проверить возможность реализации и изучить уровень гибкости, которого может достичь создаваемый язык программирования.

Классификация языков программирования

Современные языки программирования можно классифицировать по нескольким основным критериям.

1. По способу выполнения:

- компилируемые (например, C/C++);
- интерпретируемые (например, Python);
- смешанные (с использованием виртуальной машины, например, Java).

2. По парадигме программирования:

- императивные;
- функциональные;
- логические;
- объектно-ориентированные;
- мультипарадигменные.

3. По системе типов:

- статически типизированные и динамически типизированные;
- строго типизированные и слабо типизированные.

Статическая типизация позволяет выявлять ошибки на этапе компиляции или анализа, что повышает надёжность программ. В свою очередь, динамическая типизация обеспечивает большую гибкость и скорость разработки, но переносит часть ошибок на этап выполнения.

Таким образом, выбор системы типов является компромиссом между безопасностью и удобством разработки.

Популярные языки и их особенности

Рассмотрим наиболее распространённые языки программирования и их характерные черты.

- **C/C++** — обеспечивают высокую производительность и контроль над ресурсами, однако обладают сложным синтаксисом. Наличие небезопасных конструкций (например, работа с памятью) приводит к частым ошибкам, а диагностические сообщения компилятора могут быть трудны для понимания.
- **Java** — предлагает строгую объектную модель, переносимость и хорошо структурированный код. Однако язык характеризуется избыточностью синтаксиса,

что увеличивает объём кода.
- **Python** — отличается простотой и лаконичностью, что делает его популярным для быстрого прототипирования. При этом язык уступает по производительности и не обеспечивает полноценной статической проверки типов.
- **Go** — сочетает строгий синтаксис и простоту, а также предоставляет удобные средства для разработки параллельных программ. Однако язык не всегда позволяет выразить сложные конструкции компактно.
- **Rust** — ориентирован на безопасность работы с памятью и предотвращение ошибок на этапе компиляции. Главным недостатком является высокая сложность освоения и трудность реализации комплексных структур.

Отдельно следует отметить, что такие языки, как C/C++, исторически развивались длительное время. Добавление нового функционала часто приводит к усложнению синтаксиса и появлению неоднородных решений, что затрудняет их использование и реализацию компиляторов.

Язык Python, напротив, получил широкое распространение благодаря простоте. Однако отсутствие встроенной строгой статической типизации привело к появлению внешних инструментов, таких как mypy, предназначенных для анализа типов.Это указывает на востребованность подобных механизмов, которые, по возможности, должны быть встроены в сам язык.

Недостатки языков

Анализ существующих языков позволяет выделить ряд общих недостатков:

- наличие конструкций, подверженных ошибкам (например, ручное управление памятью в C++);
- избыточность и громоздкость синтаксиса (Java, C++);
- высокая сложность освоения (C++, Rust);
- ограниченная производительность интерпретируемых языков (Python);
- недостаточная выразительность или лаконичность некоторых конструкций;
- отсутствие встроенных средств статического анализа в ряде языков.

Данные недостатки формируют основу для разработки нового языка, ориентированного на простоту, безопасность и выразительность.

Выводы по результатам анализа

Проведённый анализ показывает, что ни один из существующих языков не сочетает в полной мере следующие характеристики:

- простоту синтаксиса

- строгую и удобную систему типов
- высокую производительность
- встроенные средства статического анализа
- лаконичность записи конструкций

Это подтверждает актуальность изучения инструментов разработки компиляторов и интерпретаторов, а также подталкивает к дальнейшему изучения вопроса о создании нового языка программирования.

Формулировка требований функционала к инструментам разработки языков

На основе проведённого анализа можно сформулировать требования к разрабатываемому языку и инструментам его реализации:

1. **Поддержка статического анализа:**
   - автоматическое определение типов переменных и функций
   - проверка корректности возвращаемых значений
   - анализ полноты ветвлений и наличия `return`
2. **Простота и лаконичность синтаксиса:**
   - минимизация служебных конструкций
   - поддержка сокращённых форм записи
3. **Безопасность выполнения:**
   - нахождение потенциально опасных конструкций на этапе анализа
   - контроль корректности операций
   - соответствие типов данных
4. **Гибкость конструкций управления:**
   - расширенные и удобные формы циклов (в частности, различные варианты `for`);
   - поддержка выразительных управляющих структур
5. **Удобство реализации интерпретатора:**
   - формализуемая грамматика
   - возможность автоматической генерации лексера и парсера
   - простота построения синтаксического дерева

Таким образом, сформированные требования определяют направление разработки языка Aqua и его интерпретатора, ориентированного на сочетание простоты, выразительности и надёжности.

Архитектура интерпретатора

Общая схема компилятора

Работа любого языка программирования начинается с считывания текста, представляющего собой исходный код программы, и последующее преобразование текста в удобную для анализа структуру. Зачастую этой структурой является дерево разбора. Это сложный процесс, поэтому его разбивают на несколько этапов, за которые отвечают разные подпрограммы. В общем случае порядок работы современного компилятора или интерпретатора выглядит следующим образом: Исходный код, лексический анализ (Lexer), синтаксический анализ (Parser), семантический анализ, генерация исполняемого файла / интерпретация.

Лексический анализ

Первым этапом обработки исходного кода программы является лексический анализ. Можно сказать, что исходный код представляет собой массив символов. Для более удобной обработки, его необходимо разбить на части. Простое разделение по пробелам не подойдет, так как в программном коде, в отличии от обычного текста, есть арифметические и специальные символы, между которыми может даже не быть пробелов. Поэтому, для правильного разделения входных символов, на практике используют регулярные выражения.
Для примера рассмотрим следующий код:
`int x = 10;`
Данный код отвечает за создание целочисленной переменной `x` со значением `10`.
Разбив данную строку, мы получим:
`'int', 'x', '=', '10', ';'`
В контексте лексического анализа данные последовательности символов называют **лексемы**.
Рассмотрим аналогичный фрагмент кода, который создаёт переменную с плавающей точкой:
`float num = 2.4;`
Если разбить его на лексемы, получим:
`'float', 'num', '=', '2.4', ';'`
В данных двух примерах мы можем заметить сходство: первым пишется тип переменной, затем название, знак равно, числовое значение, знак точка с запятой.
Запишем это в более общей форме:
`<тип 'float'> <название 'num'> <знак '='> <число '2.4'> <знак ';'>`
Каждый такой элемент называется **токеном**. Он представляет собой категорию символа и хранит в себе исходный текст (лексему). Используя принятые обозначения, которые часто используются на практике и в литературе, запишем это следующим образом:
`<KEYWORD 'int'>, <ID 'x'>, <OPERATOR '='>, <NUMBER '10'>, <KEYWORD ';'>` и `<KEYWORD 'float'>, <ID 'num'>, <OPERATOR '='>, <NUMBER '2.4'>, <KEYWORD ';'>`.
Зарезервированные в языке символы называют KEYWORD, названия переменных или функций - ID или IDENTIFIER, арифметические знаки OPERATOR, а числовые значения - NUMBER.
Токены могут включать в себя дополнительную информацию, такую как: категорию лексемы, номер строки, номер столбца, имя файла.

Обычно, чтобы разбить исходный текст на набор токенов, используют регулярные выражения (regex). Например, `IDENTIFIER` можно представить регулярным выражением `[a-zA-Z_][a-zA-Z0-9_]*`, а NUMBER представить как `[0-9]+`.

Лексические ошибки

Ошибка на этапе лексического анализа возникает при несоответствии входного символа ни одному из определений лексем.
Например, код:
`string 2s = "Hello world";`
вызовет ошибку, поскольку название переменной (`IDENTIFIER`) не может начинаться с цифры. Также лексические ошибки могут возникать при использовании специфичных символов или кодировок.

Заключение

Лексический анализ — это начальный этап обработки, на котором исходный код разбивается на последовательность лексем и преобразуется в набор токенов. Этот процесс позволяет структурировать текст программы и подготовить его для дальнейшего синтаксического анализа.
За разбиение на токены и поиск ошибок отвечает лексический анализатор (лексер), используя регулярные выражения.
В современных языках программный код лексера обычно не пишут вручную. Для этого используют кодогенерацию на основе заранее прописанной лексической грамматики.
Один из самых популярных генераторов лексеров - "flex", написанного на C++, от которого позже пошли аналоги для разных языков программирования: jflex, todo.
Для генерации нужно передать файл, в котором прописаны название токена и соответствующее регулярное выражение для поиска.

Синтаксический анализ

Понятие синтаксического анализа

Синтаксический анализ (парсинг) — это этап обработки исходного кода, на котором набор токенов преобразуется в структурированное представление, которое называется абстрактным синтаксическим деревом (AST).
На этом этапе проверяется соответствие программы правилам грамматики, выявляются ошибки, строится синтаксическое дерево.

Понятие грамматики

Ещё раз рассмотрим ранее упомянутые примеры:
Набор токенов для строки `int x = 10;`:
`<KEYWORD 'int'>, <ID 'x'>, <OPERATOR '='>, <NUMBER '10'>, <KEYWORD ';'>`
И для строки `float num = 2.4;`:
`<KEYWORD 'float'>, <ID 'num'>, <OPERATOR '='>, <NUMBER '2.4'>, <KEYWORD ';'>`.

Далее для удобства будем записывать `KEYWORD 'float'` как `FLOAT`, `<KEYWORD 'int'>` `как` INT, <OPERATOR '='> `как` EQUAL, <KEYWORD ';'> `как SEMICOLON. Можно заметить, что полученные последовательности довольно схожи. Один из базовых подходов в программировании и в разработке программного обеспечения - это абстракция и обобщение. Здесь мы тоже воспользуемся этим методом. В общем виде инициализация переменной (variable initializaton) будет выглядеть так:` TYPE ID EQUAL NUMBER SEMICOLON `Тип переменной - это TYPE, название переменных, как мы уже говорили, принято называть ID, оператор присваивания - EQUAL, число для инициализации - NUMBER, а точка с запятой - SEMICOLON. Большинство элементов здесь представляют собой простые токены, но TYPE является составным, поскольку состоит из FLOAT, либо INT. Запишем для него определение:` TYPE: FLOAT | INT `Знак` : `означает 'является', а знак` |` означает 'или'.
В теории формальных языков и грамматик, данная запись называется правилом производства (продукция), где атомарные элементы (токены) называются терминалами, составные - нетерминалами.
Первый нетерминал, с которого начинается самое первое правило производства, называется начальным символом и обозначается он как S или Start.
Грамматика является набором продукций, задающие правила построения допустимых цепочек символов.

Таким образом, в примере выше, FLOAT и INT являются терминалами, а TYPE - нетерминалом.

Усложним наш пример:
`int n = 2 + 2 - 1`
Теперь знак точки с запятой в конце является необязательным, а вместо числа можно написать арифметическое выражение без скобок со знаками плюс или минус.
Тогда фрагмент грамматики для инициализации переменной будет выглядеть следующим образом:

```
VAR_INIT: TYPE ID ASSIGN EXPRESSION LINE_END
TYPE: FLOAT | INT
EXPRESSION: NUMBER MATH_OP EXPRESSION | NUMBER
MATH_OP: PLUS | MINUS
LINE_END: SEMICOLON |
```

VAR_INIT - это продукция инициализации переменной, TYPE - тип, EXPRESSION - арифметическое выражение, MATH_OP - математический оператор плюс или минус, LINE_END - конец строки.
В данном примере нетерминал LINE_END порождается терминалом SEMICOLON или может вовсе отсутствовать. Нетерминал EXPRESSION в своём определении имеет рекурсивный вызов, что является ключевым принципом в построении грамматик и позволяет описывать конструкции произвольной длины.

Как и код лексера, код синтаксического анализатора (парсера) генерируется на основе грамматики, описанных в таком же формате, как было описано в примерах выше.

Одним из самых популярных инструментов для генерации парсера является byacc. На его основе также существуют аналоги для других языков программирования: jbyacc, todo и т.д. Таким образом, создание любого языка сводится к описанию грамматики, то есть правила производства.

Классификация грамматик

В классификации, предложенной Ноаном Хомским, грамматики делят на 4 вида.

Неограниченные грамматики

Это общий класс грамматик, где все остальные являются его частными случаями.
Правила имеют вид:
$\alpha \rightarrow \beta$
где:

- α — непустая строка символов
- β — любая строка
  Такие грамматики используют редко, Поскольку в таких грамматиках есть возможность поместить несколько нетерминалов в левой части правила производства, они обладают высокой сложностью.

Контекстно-зависимые грамматики

Правила имеют вид:
$\alpha A \beta \rightarrow \alpha \gamma \beta$
Где:

- α, β — любая строка символов
- A — нетерминал
- γ — непустая строка
  Как следует из названия, применения правил производства в таких грамматиках зависит от окружения. Если говорить более простым языком, происходит замена A на γ. Особенностью является то, что длинна получившейся строки после применения продукции не должна уменьшается.
  Данные грамматики всё ещё обладают высокой сложностью, поэтому применяются ограниченно.

Контекстно-свободные грамматики

Правила имеют вид:
$A \rightarrow \gamma$
где:

- A — нетерминал
- γ — любая строка символов
  Поскольку в левой части производства находится всего один нетерминал,

контекстно-свободные грамматики обладают не такой высокой сложностью и легко обрабатываются программно.
Широко применяются для описания синтаксиса языков программирования.

Пример кс грамматики:

```
Expr → Expr + Term
Expr → Term
Term → number
```

Регулярные грамматики

Правила имеют вид:

- A→aB
- A→a
  где:
- A, B — нетерминалы
- a — терминал
  Регулярные грамматики являются наиболее простым классом грамматик.
  Широко используются в поиске и шаблонах благодаря регулярным выражениям. На этапе лексического анализа с помощью регулярных выражений описываются идентификаторы, числа, ключевые слова и так далее.

Синтаксические деревья разбора

Дерево разбора - представление исходного кода программы в виде структуры данных "дерево", узлы которого представляют собой нетерминальные и терминальные символы, и соответствуют грамматике языка.
Есть два основных вида деревьев разбора:

Полное дерево разбора

Полное дерево разбора (Parse Tree, Concrete syntax tree) полностью отражает грамматику языка и содержит все нетерминальные и терминальные символы.

Пример
Рассмотрим выражение:
2 + 3 *4*
И упрощённую грамматику:
E → *E* + *T* | *T*
T → *T* F | F
F → (E) | number
Дерево разбора будет содержать все промежуточные нетерминалы:
Картинка.

Полные деревья разбора почти не используются, поскольку содержат много избыточной информации (например, скобки, служебные правила).

Абстрактное синтаксическое дерево

Абстрактное синтаксическое дерево (AST, Abstract Syntax Tree) представляет собой упрощённое полное дерево разбора. Оно лишено ненужных деталей, а часть узлов сокращены. Остаются лишь значимые символы, которые в дальнейшем пригодятся для обработки. Такие элементы, как скобки, удаляются, поскольку порядок выполнения и принадлежность символов к конкретному выражению реализуется за счёт древовидной структуры хранения данных.
На практике, при обработке языков используют именно AST, что даёт ряд преимуществ: компактность хранения, ускорение и упрощение обхода дерева, легкость выполнения модификаций. Создание AST реализуют либо методом удаления узлов из полного синтаксического дерева, либо не значимые символы изначально игнорируются на этапе обработки цепочек грамматики.

Пример
Для того же выражения:
2 + 3 * 4

AST будет выглядеть так:
Картинка

Виды синтаксических анализаторов

Синтаксический анализатор (парсер) работает на основе **контекстно-свободной грамматики**, получает на вход результат лексического анализа, а в качестве результата выдаёт дерево разбора.
По принципу работы существует два основных вида анализаторов:

LL-парсер

Это нисходящий парсер, который:

- читает вход слева направо (**L — left-to-right**)
- строит левый вывод (**L — leftmost derivation**)

Наиболее распространённый вариант — **LL(1)**:

- использует **1 символ предпросмотра (lookahead)**

LR-парсер

LR-парсер — это восходящий парсер, который:

- читает вход слева направо (**L**)

- строит правый вывод в обратном порядке (**R**)
  LR-парсеры работают по принципу:
  свёртки (reduce) последовательности символов к нетерминалам

Основные операции:

- **shift** — сдвиг входного символа в стек
- **reduce** — замена цепочки на нетерминал
- **accept** — успешное завершение
- **error** — ошибка
  Парсер использует:
- стек
- таблицу действий (action/goto)

Существуют различные модификации:

- LR(0) — простой, но слабый
- SLR(1) — улучшенный
- LALR(1) — компромисс (используется в Yacc)
- LR(1) — самый мощный

Синтаксические ошибки

Ошибка на этапе синтаксического анализа возникает при несоответствии входного потока символов ни одному из определений правил производства грамматики.
Например, код:
`string s = "Hello" "world";`
вызовет ошибку, поскольку между двумя строками нет знака действия или оператора.
В другом примере:
`string s = "Hello world"`
можно увидеть ошибку, знакомую многим программистам - пропущен знак точка с запятой.
Стоит отметить, что на этом этапе проверяется только соответствие правилам грамматики, поэтому код `float f = "Hello" / 5;` на данном этапе будет корректным.

Заключение

Синтаксический анализатор отвечает за преобразование последовательности токенов в структурированное представление программы — абстрактное синтаксическое дерево. Именно на этом уровне формируется понимание структуры кода в соответствии с заданной грамматикой, а также выявляются синтаксические ошибки.

Грамматика выступает основой языка, позволяя описывать допустимые конструкции через правила производства.

Выбор конкретного метода синтаксического анализа (LL или LR) зависит от требований к языку и сложности грамматики, однако оба подхода широко применяются в современных программах.

Таким образом, синтаксический анализатор стоит дерево разбора, которое за тем передаёт на следующий этап обработки - семантический анализ.

Семантический анализ

После построения абстрактного синтаксического дерева следующим этапом является семантический анализ. Его цель — выявить смысловые ошибки и дополнить дерево необходимой информацией посредством его обхода и анализа.

Таблица символов

Таблица символов — это структура данных, которая хранит информацию об идентификаторах в программе: имя, тип, значение, ссылка на экземпляр родительского элемента и другие дополнительные атрибуты.

Проверка типов

Проверка типов включает в себя проверку совместимости типов в выражениях, в присвоениях, передаваемых аргументах функции и возвращаемых значений функции. В выражениях типы данных проверяются, основываясь на типе, указанном в таблице символов и на основе литералов, которые строго задают тип значения. В большинстве языков программирования допускается автоприведение типов, когда итоговое значение отдаётся по приоритету. Например, при сложении чисел с типами данных float и int результат будет float, поскольку у него приоритет больше.

Проверка инициализации

Во время обхода дерева разбора, при обработке узла с оператором присвоения значения, анализатор взаимодействует таблицу символов. Если это первая инициализация идентификатора - добавляется новая запись в таблицу символов, если повторная - вызывается ошибка. При присвоении значения к идентификатору проверяется, чтобы значение уже присутствовало в таблице символов, то есть уже было инициализировано ранее.

Проверка областей видимости

Область видимости определяет, в каком месте программы и к какому именно идентификатору осуществляется доступ.
Основные виды областей видимости:

- глобальная (за пределами функций)
- локальная (внутри функций)

- блочная (внутри блоков if, while и подобных)
  Зачастую допускается объявление идентификаторов с одинаковым именем в разных или вложенных областях видимости.
  Один из способов проверки их корректности осуществляется с помощью иерархической структуры в таблице символов. То есть при обращении к идентификатору в таблице символов ищется записать с соответствующим именем, если такой записи не нашлось, то поиск продолжается в родительской таблице.

Аннотирование AST

После выполнения проверок, для упрощения последующей обработки, в AST добавляется дополнительная информация, такая как:

- типы выражений
- ссылки на записи в таблице символов
- информация о преобразованиях типов

Семантические ошибки

Ошибка на этапе семантического анализа возникает при несоответствии смысловой корректности программы требованиям языка.
К таким несоответствиям относятся:

- несоответствие типов данных
- использование необъявленных идентификаторов
- повторное объявление переменных в одной области видимости
- несоответствие количества или типов аргументов функции
- применение неподдерживаемых операторов

Например, код:
`int s = "Hello world";`
вызовет ошибку, поскольку к переменной с типом int присваивается значение с типом string.

Заключение

Семантический анализ является важным этапом обработки программы, потому как от него зависит преждевременное нахождение смысловых ошибок, связанных с неправильным использованием конструкций языка, таких как несоответствие типов, нарушение областей видимости и некорректное обращение к идентификаторам. Степень сложности обнаруживаемых ошибок, ограничивается только реализацией анализатора и может включать и более сложные сценарии.

Результатом данного этапа является абстрактное синтаксическое дерево, дополненное информацией необходимой для дальнейшего выполнения программы и её оптимизации.

Интерпретация

Следующим этапом обработки является использование полученного абстрактного синтаксического дерева. Строго говоря, существует лишь два последующих пути обработки: генерация кода (машинного кода или кода другого языка программирования) и выполнение программы (интерпретация).
В данном исследовании будет рассмотрен процесс интерпретации.
Используя алгоритм обхода дерева в глубину, программа посещает все узлы и выполняет действия в соответствии с типом узла и сохранённой в нём информацией.

Обход дерева чаще всего реализуется рекурсивно. Для каждого типа узла определяется соответствующая логика выполнения:

- **выражения** вычисляются и возвращают значение;
- **операторы присваивания** изменяют состояние таблицы символов;
- **условные конструкции** выбирают ветку выполнения;
- **циклы** повторяют выполнение поддерева до выполнения условия выхода.

В процессе интерпретации ключевую роль играет **таблица символов**. При обработке узлов дерева интерпретатор обращается к этой таблице для чтения и обновления значений. Именно в ней хранится состояние всей программы.

Ошибки во время выполнения

В данных класс ошибок попадают все остальные ошибки, которые не удалось выявить на предыдущих этапах. Чаще всего это ошибки, связанные со значениями самих переменных.

Например, код:

```
int a = 24;
int b = a / 2 - 12;
int r = 5 / b;
```

вызовет ошибку "деление на ноль". На этапе семантического анализа подобные ошибки трудно выявить, поскольку они требуют исполнения кода программы для определения значений переменных, но зачастую проверки и вовсе невозможны, поскольку данные приходят из вне.

Заключение

Важно отметить, что интерпретатор выполняет программу на ходу, без предварительной генерации машинного кода. Это упрощает реализацию и повышает переносимость, однако уступает компилируемым программам в производительности.

Практическая часть

Выбор инструментов и технологий

Для реализации интерпретатора языка программирования Aqua был выбран ряд инструментов, обеспечивающих производительность, удобство разработки и расширяемость системы.

В качестве основного языка реализации выбран Java. Данный выбор обусловлен рядом факторов. Java имеет богатую стандартную библиотеку, относительно простой синтаксис, автоматическое управление памятью, следует объектно-ориентированной парадигме. Всё это упрощает структурирование компонентов интерпретатора таких как: лексический анализатор, парсер, семантический анализатор, внутренние структуры данных интерпретатора. Также, Java работает на виртуальной машине, что позволяет запускать код на разных операционных системах без перекомпиляции. Дополнительным преимуществом является наличие развитых инструментов разработки. В частности, для Java широко используется профессиональная интегрированная среда разработки IntelliJ IDEA, которая предоставляет мощные средства навигации по коду, рефакторинга, отладки и анализа программ. Использование данной среды существенно повышает продуктивность разработки и упрощает сопровождение проекта.

Для реализации лексического анализа используется инструмент JFlex. Он представляет собой генератор лексических анализаторов, аналогичный утилите Lex, но ориентированный на Java. Инструмент позволяет описывать правила токенизации с использованием регулярных выражений, автоматически генерируя эффективный код. Это уменьшает время разработки и количество ошибок, в сравнении с ручной реализацией.

Синтаксический анализ реализован с использованием BYACC/J. Данный инструмент является адаптацией классического генератора парсеров YACC для языка Java. Он позволяет задавать контекстно-свободную грамматику языка и автоматически генерировать LALR(1)-парсер. Использование данного инструмента упрощает процесс разработки и уменьшает количество ошибок.

Для визуализации синтаксических деревьев применяется Graphviz. Этот инструмент используется для построения и отображения графовых структур, таких как абстрактные синтаксические деревья (AST). Визуализация дерева разбора позволяет упростить отладку интерпретатора, а также наглядно продемонстрировать структуру обрабатываемых программ.

Таким образом, выбранные инструменты позволяют эффективно реализовать интерпретатор, обеспечивая удобство разработки, отладки системы и избавляя от части потенциальных ошибок.

Структура программы интерпретатора

Программная реализация интерпретатора языка Aqua организована в виде модульной структуры, разделённой на два основных уровня: фронтенд (анализ исходного кода) и рантайм (выполнение программы). Все исходные файлы располагаются в директории `/src`, что соответствует стандартной практике организации Java-проектов.

Основной файл
Точкой входа в приложение является файл `Main.java`. Данный класс содержит минимальную логику и выполняет делегирование управления методу `main()` класса `j0`. Такой подход позволяет изолировать пользовательский интерфейс запуска программы от внутренней логики интерпретатора и упрощает дальнейшую модификацию архитектуры.

Директория frontend — лексический и синтаксический анализ
Отвечает за обработку исходного текста программы и преобразование его в синтаксическое дерево.

- `j0.java` — центральный класс фронтенда. Он координирует процесс анализа: инициализирует лексер, запускает парсер и управляет потоком обработки входных данных.
- `Yylex.java` — лексический анализатор, сгенерированный с помощью JFlex. Выполняет разбиение входного потока символов на последовательность токенов.
- `Parser.java` — синтаксический анализатор, сгенерированный с использованием BYACC/J. На основе заданной грамматики строит дерево разбора.
- `Token.java` — структура, представляющая токен. Содержит тип токена, его текстовое представление и номер строки, что необходимо для диагностики ошибок.
- `Tree.java` — реализация узла синтаксического дерева. Включает символ грамматики, список дочерних узлов и связанные токены.
- `ParserVal.java` — вспомогательная структура, используемая парсером для передачи значений между правилами грамматики.
- `Serial.java` — генератор уникальных идентификаторов, применяемый для маркировки узлов дерева и других объектов.
- `Yyerror.java` — модуль обработки синтаксических ошибок, обеспечивающий информативные сообщения при некорректном вводе.

Таким образом, пакет `frontend` реализует полный цикл преобразования исходного текста программы в абстрактное синтаксическое дерево.

Директория runtime — интерпретация и выполнение
Пакет `runtime` реализует этап семантического анализа и непосредственного выполнения программы.

- `Interpreter.java` — ключевой компонент системы. Выполняет обход синтаксического дерева, проводит семантический анализ и интерпретирует конструкции языка.

- `SymbolTable.java` — реализация таблицы символов. Обеспечивает хранение переменных, их значений и поддерживает вложенные области видимости (scope).
- `RuntimeValue.java` — универсальная обёртка для значений времени выполнения, содержащая тип и соответствующее значение.
- `ValueType.java` — класс с набором констант, определяющих возможные типы данных в языке (например, `int`, `string` и др.).
- `PrimitiveHandler.java` — модуль обработки примитивных операций, таких как арифметические вычисления и операции сравнения.
- `FunctionHandler.java` — отвечает за обработку вызовов функций, включая передачу аргументов и возврат значений.
- `FunctionType.java` — структура, описывающая сигнатуру функции (параметры, тип возвращаемого значения).
- `ErrorHandler.java` — компонент обработки ошибок времени выполнения, обеспечивающий корректное завершение программы и вывод диагностической информации.

Описание языка программирования Aqua

Синтаксис и семантика языка

Язык программирования Aqua представляет собой процедурно-ориентированный интерпретируемый язык с синтаксисом, близким к языкам семейства C/Java, но упрощённым для учебных целей. Программа на Aqua состоит из набора операторов и функций, где точкой входа обычно является функция `main`.

Синтаксис языка характеризуется следующими особенностями:

- использование фигурных скобок `{}` для определения блоков кода;
- опциональное использование точки с запятой `;` в конце выражений;
- возможность записи условий без обязательных круглых скобок;
- поддержка однострочных конструкций без явного блока.

Идентификаторы формируются по стандартным правилам: начинаются с буквы или символа подчёркивания и могут содержать буквы, цифры и `_`. Язык чувствителен к регистру.

Семантика языка основана на интерпретации абстрактного синтаксического дерева. Все выражения вычисляются последовательно, а управление потоком осуществляется через стандартные конструкции (условия, циклы, функции). Области видимости реализуются с помощью вложенных таблиц символов, что позволяет корректно обрабатывать локальные переменные и параметры функций.

Типы данных и конструкции управления

Aqua поддерживает ограниченный, но достаточный для исследовательских задач набор типов данных:

- `int` — целые числа;
- `float` — числа с плавающей точкой;
- `bool` — логические значения;
- `intarray` — массивы целых чисел;

Типизация статическая: тип переменной задаётся при объявлении или вычисляется автоматически и должен соблюдаться при использовании.

Язык поддерживает стандартный набор операций:

- арифметические (`+`, `-`, `*`, `/`, `%`);
- логические (`&&`, `||`, `!`);
- операции сравнения (`<`, `>`, `==`, `!=` и др.);
- операции присваивания и сокращённые формы (`+=`, `++` и др.).

Ключевые конструкции управления:

- **Условные операторы**: `if`, `else if`, `else`;
- **Циклы**:
  - `while` и `do-while`;
  - расширенный `for` с несколькими вариантами синтаксиса (включая диапазоны и шаг);
- **Управляющие операторы**: `break`, `continue`, `return`.

Функции объявляются с помощью ключевого слова `fn` и могут принимать параметры и возвращать значения. Поддерживается рекурсия.

Особенности и отличия от других языков

Язык Aqua обладает рядом особенностей, отличающих его от распространённых языков программирования:

1. **Упрощённый синтаксис**
   - необязательные точки с запятой;
   - опциональные скобки в условиях;
   - опциональный тип данных при объявлении переменной
   - компактные формы записи циклов.
2. **Расширенный синтаксис цикла for**
   - поддержка нескольких форм записи (короткая, диапазонная, полная);
   - возможность задания шага и границ в декларативной форме.

3. **Автоопределение типов**
    - Возвращаемый тип функции определяется автоматически
    - Тип при инициализации определяется за счёт литералов

По сравнению с Java язык Aqua проще и менее формализован (например, отсутствует обязательная объектная модель). В отличие от Python, Aqua использует статическую типизацию и фигурные скобки вместо отступов. По сравнению с C/C++ отсутствуют указатели и ручное управление памятью.

Ограничения языка включают отсутствие:

- объектно-ориентированного программирования;
- обработки исключений;
- обобщённых типов (generics);
- развитой стандартной библиотеки.

Примеры программ на Aqua

Ниже приведены типовые примеры программ, демонстрирующие возможности языка.

Простейшая программа:

```
println("Hello, World!")
```

Условный оператор:

```
int temperature = 30

if temperature >= 0 {
    println("Тепло")
} else {
    println("Холодно")
}
```

Цикл for:

```
for i : 5 {
    println(i)
}
```

Функция (рекурсия):

```
fn factorial(int n) {
    if n <= 1 {
        return 1
    }
    return n * factorial(n - 1)
}

println(factorial(5))
```

Работа с массивами:

```
intarray arr = newIntArray(5)

for i : getLen(arr) {
    setElement(arr, i, i * 2)
}

println(arr)
```

Представленные примеры демонстрируют основные возможности языка: работу с типами данных, использование управляющих конструкций, определение функций и обработку массивов.

Реализация лексического анализатора

Описание лексем

Правила разбора на лексемы задаются в файле `javalex.l` с использованием регулярных выражений.

Основные группы токенов:

1. Ключевые слова

```
"int", "float", "if", "else", "while", "for", "fn", "return"
```

2. Идентификаторы

```
[a-zA-Z_][a-zA-Z0-9_]*
```

3. Числовые литералы

```
[0-9]+
[0-9]*"."[0-9]*([eE][+-]?[0-9]+)?
```

4. Строковые литералы

```
\"([^\"]|(\\.))*\"
```

5. Операторы

- арифметические: `+`, `-`, `*`, `/`;
- логические: `&&`, `||`;
- сравнения: `==`, `!=`, `<`, `>` и др.;
- составные: `+=`, `++`, `--`.

6. Игнорируемые элементы

- пробельные символы;
- комментарии (`//`, `/* */`).

Каждому регулярному выражению соответствует код, который должен выполниться при считывании — вызов метода `scan()` с передачей кода токена или выполнение вспомогательных функций, например, увеличить номер строки.

Класс лексера

Лексер в интерпретаторе Aqua реализован с использованием генератора JFlex, который на основе регулярных выражений, формирует класс `Ylex.java`.
Основным методом класса является `yylex()`. Данный метод считывает символы из входного потока, сопоставляет их с шаблонами регулярных выражений и для каждой лексемы выполняет код, который был указан в файле `javalex.l`.

Создание токенов

После перехода из метода `yylex()` в метод `scan()` класса `j0`, начинается создание объекта класса `Token`. В конструктор этого класса передаются: код токена, переданный в виде параметра; текст лексемы, получаемый из метода `Ylex.yytext()`; номер строки - `Ylex.yylineno`. Затем токен передаётся парсеру через поле `j0.par.yylval`.

Обработка ошибок

Лексический анализатор включает базовую обработку ошибок. В случае обнаружения неизвестного символа вызывается метод `yyerror` класса `Yyerror`, формирующий сообщение об ошибке.
Пример ошибки:

```
Lexer error: unrecognized character '@' at line 1
```

Заключение

Подводя итог, процессы на данном этапе можно описать следующим образом: Сперва исходный файл считывается как поток символов, лексер последовательно анализирует входной поток, затем распознанные последовательности символов преобразуются в токены и, наконец, полученные токены передаются синтаксическому анализатору.

Реализация синтаксического анализатора

Описание грамматики

Правила синтаксического разбора задаются в файле `j0gram.y`. В фигурных скобках после правил прописаны семантические действия, которые указывают, какие из токенов попадут в дерево разбора, а какие будут проигнорированы.

Основные группы конструкций:

1. Описание блоков кода и методов

```
Stmt: Block | ';' | ExprStmt | BreakStmt | ReturnStmt
      | IfStmt | IfElseStmt | MethodDecl
      | DoWhileStmt | WhileStmt | ForStmt | LocalVarDeclStmt;

MethodDecl: FN IDENTIFIER '(' FormalParmList ')' Block { ... }
```

2. Описание переменных и типов

```
FieldDecl: Type VarDecls { ... }
Type: INT | FLOAT | BOOL | STRING | Name | INTARRAY
```

3. Операторы и выражения

- арифметические, логические, сравнения, присваивания, вызовы функций и др.

Класс парсера

С помощью BYACC/J на основе файла грамматики автоматически генерируется класс `Parser.java`. Ключевым методом является `yyparse()`. Внутри себя данный метод вызывает функцию `yylex()` для получения следующего для обработки токена, а затем сопоставляет значение с заданными правилами производства. Для передачи значений между правилами используется класс `ParserVal`, который, по своей сути, является контейнером для объекта `Token`. Токены помещаются в объекты класса `Tree`, которые реализуют узлы дерева разбора.

Обработка ошибок

При обнаружении синтаксической ошибки вызывается метод `yyerror` класса `Yyerror`, формирующий сообщение с указанием строки и типа ошибки.

Пример ошибки:

```
Parser error: unexpected token '}' at line 10
```

Заключение

Сформированные на предыдущем этапе токены передаются парсеру, который, используя правила производства указанные в файле грамматики `j0gram.y`, строит синтаксическое дерево и сообщает об ошибках.
Класс `j0.java`, в свою очередь, выполняет роль координатора, вызывая методы обоих анализаторов.

Реализация семантического анализатора

Полученное на предыдущем этапе абстрактное синтаксическое дерево подлежит анализу на предмет ошибок и для вычисления служебной информации. Этот функционал реализует метод `semantic()` класса `Interpreter`. В качестве параметров он принимает дерево разбора и глобальную область видимости, в которой уже заданы некоторые значения, например, встроенные функции. Работа семантического анализатора, начинается с вызова `analyzeBlock` для корня дерева разбора.
Данный метод рекурсивно обходит дерево, выполняя ряд операций:

- Объявления переменных: добавляет новые записи в `SymbolTable`, проверяет их на повтор.
- Присваивания: проверяет существование переменной, типы, допустимость операций.
- Анализирует выражения: вызывает функцию `analyzeExpr()`, которая вычисляет тип результата, вызывает обработчики операций.
- Вызовы функций: проверяет наличие, количество и типы аргументов, создает новую область видимости.
- Вложенные блоки: создает вложенные таблицы символов.

В основе реализации `SymbolTable` стоит структура данных словарь, где ключом является имя переменной, которому соответствует её текущее состояние. В Java такая структура объявляется как `HashMap<String, RuntimeValue>`. `RuntimeValue.java` представляет собой универсальный контейнер для значений разных типов (int, float, bool, массивы, функции), который содержит тип переменной и само значение.
Помимо этого таблица символов поддерживает вложенность областей видимости через ссылку на родителя, а также реализует следующие методы:

- `addVar(String name, RuntimeValue value)`: добавляет переменную, проверяет на повтор.
- `getVar(String name)`: ищет переменную в текущей и родительских областях.

- `contains(String name)` : проверяет наличие переменной.
  По завершении обработки, ссылка на `SymbolTable` сохраняется в соответствующем узле самого синтаксического дерева.

Класс `PrimitiveHandler` отвечает за проверку типов операндов, автоматическое приведение к нужному типу и реализацию базовых операций над типами, а именно: арифметические, логические, сравнение.

Обработка функций осуществляется в классе `FunctionHandler`. При вызове функции он находит её в таблице символов, проверяет аргументы, создает новую область видимости, выполняет тело функции. Также класс имеет методы для обработки встроенных функций, такие как: добавление их в область видимости и вызов их реализации, написанной на Java.

Обработка ошибок

Обработка ошибок находится `ErrorHandler`. При возникновении нештатной ситуации вызывается один из методов класса, который, в свою очередь, выбрасывает исключение, обработка которого находится на уровень выше, в классе `j0`.

Заключение

Семантический анализатор гарантирует корректность программы на уровне типов, областей видимости, операций и вызовов функций, предотвращая ошибки времени выполнения.

Выполнение кода

Метод `eval()` класса `Interpreter` отвечает за вычисления узлов дерева и выполнение исходной программы.
Как и в семантическом анализе, здесь используется схожий принцип обхода дерева разбора, но вместо анализа узлов дерева происходит их исполнение.
Выполняет различные действия в зависимости типа обрабатываемого узла:

- **Блоки**: создает новую область видимости, интерпретирует все инструкции по порядку.
- **Присваивания**: вычисляет выражение, сохраняет результат в таблице символов.
- **Выражения**: вызывает `evalExpr` которое, возвращает вычисленное значение.
- **Вызовы функций**: вызывает `FunctionHandler.eval`, передает аргументы, создает новую область видимости для локальных переменных.

Работа с функциями

Пользовательские функции в классе `FunctionHandler` обрабатываются следующим образом:

- При вызове создается новая область видимости.

- Аргументы копируются в локальные переменные.
- Последовательно интерпретируются инструкции тела функции.
- Возвращаемое значение передается обратно.

Для встроенных функций вызывается реализация, написанная на языке Java.
Перечень встроенных функций и их назначение:

- print(...) — выводит значения аргументов в одну строку через пробел.
- println(...) — выводит значения аргументов в одну строку через пробел и переводит строку.
- printChar(int code) — выводит символ по его числовому коду.
- readInt() — читает целое число с ввода.
- readFloat() — читает число с плавающей точкой с ввода.
- newIntArray(int size) — создает новый массив целых чисел заданной длины.
- readCharsAsIntArray() — читает все строки ввода, возвращает массив кодов символов.
- readChar() — читает один символ с ввода, возвращает его числовой код.
- getElement(intarray arr, int index) — возвращает элемент массива по индексу.
- setElement(intarray arr, int index, int value) — устанавливает значение элемента массива по индексу.
- getLen(intarray arr) — возвращает длину массива.
- getArg(int index) — возвращает аргумент командной строки как массив кодов символов.
- readFile(intarray path) — читает файл по пути (массив символов), возвращает содержимое как массив кодов символов.

Обработка ошибок

Для обработки ошибок, как и в семантическом анализе, используется класс `ErrorHandler`.

Заключение

Данный этап является финальным в последовательности обработки исходного кода. Процесс интерпретации начинается с корневого узла дерева, во время которого рекурсивно выполняются все конструкции программы, изменяется состояние таблиц символов и в поток вывода выводится результат.
Стоит отметить, что на данном этапе не производится проверка инициализации переменных, областей видимости и типов. Благодаря семантическому анализу обеспечивается корректная работа программы без необходимости вычислительных затрат на проверку ошибок.

Тесты

В ходе разработки интерпретатора были реализованы юнит тесты на языке Java, используя библиотеку JUnit5. Код тестов находятся в файле `tests/main/InterpreterTests.java`. Тестируемые скрипты на языке aqua находятся в папке `tests/files/`.

Для избегания повторения кода в тестах реализованы две служебных функции:

- **test_success(String filename, Object var_value):**
  1. Загружает и запускает скрипт из файла.
  2. После интерпретации извлекает значение переменной `res` из глобальной области видимости.
  3. Проверяет, что код завершения — 0, и результат совпадает с ожидаемым.
- **test_fail(String filename, String error_msg):**
  1. Запускает скрипт и ожидает ошибку во время выполнения.
  2. Проверяет, что сообщение об ошибке содержит ожидаемое сообщение.

Итог

Тесты автоматически проверяют как успешные сценарии, так и обработку ошибок, ускоряя разработку программы.

Заключение

Оценка производительности языка

Для оценки производительности интерпретатора языка Aqua было проведено экспериментальное сравнение времени выполнения программы `examples/ege_task.aqua` с аналогичной реализацией на языке Python.
Измерения показали, что среднее время выполнения составило: 164 мс для языка Aqua и 38 мс для Python.
Таким образом, интерпретатор Aqua уступает по скорости выполнения примерно в 4 раза.

Полученный результат является ожидаемым и объясняется рядом факторов: отсутствием оптимизаций, характерных для зрелых интерпретаторов, а также учебной направленностью реализации. При этом достигнутый уровень производительности можно считать удовлетворительным, поскольку он подтверждает корректность архитектурных решений и работоспособность интерпретатора.

Достигнутые цели

В рамках данной работы была достигнута основная цель — анализ инструментов разработки языков программирования и создание интерпретатора учебного языка Aqua.

В теоретической части рассмотрены современные языки программирования, их особенности, а также принципы построения компиляторов и интерпретаторов. Отдельное внимание уделено архитектуре создания языков, включая этапы лексического, синтаксического и семантического анализа.

Обоснован выбор инструментов разработки. Использование Java в качестве основного языка реализации обеспечило переносимость, удобство разработки и наличие развитой экосистемы. Применение JFlex и BYACC/J позволило автоматизировать процессы генерации лексического и синтаксического анализаторов, что снизило вероятность ошибок и ускорило разработку.

В практической части реализованы все основные компоненты интерпретатора:

- лексический анализатор, выполняющий разбиение входного потока на токены;
- синтаксический анализатор, строящий абстрактное синтаксическое дерево;
- семантический анализатор, проверяющий корректность программы;
- выполнение кода на основе абстрактного синтаксического дерева.

Разработан и формально описан язык программирования Aqua, определены его синтаксис, семантика, типовая система и управляющие конструкции. Реализована поддержка базовых типов данных, операторов управления потоком, функций, рекурсии и целочисленных массивов.

Проведено тестирование интерпретатора с использованием модульных тестов, что позволило подтвердить корректность работы как в штатных сценариях, так и при возникновении ошибок.

Дополнительно стоит отметить, что в примерах программ на языке aqua продемонстрирована практическая выразительность языка: в файле `examples/brain_fuck_interpreter.aqua` реализован рабочий интерпретатор языка Brainfuck, что подтверждает достаточный уровень абстракции и полноту реализованных возможностей Aqua.

Перспективы развития

Перспективными направлениями являются:

- внедрение более глубокого анализа корректности программ и поиска потенциальных ошибок;
- расширение типовой системы;
- улучшение механизмов обработки ошибок;
- добавление новых структур данных и языковых конструкций.

Отдельный интерес представляет переход от интерпретации к компиляции с использованием таких инструментов, как LLVM. Это позволит повысить производительность и приблизить язык к промышленным решениям.

Выводы

В ходе работы был выполнен комплексный анализ подходов к разработке языков программирования и реализован полноценный интерпретатор учебного языка.

Практическая реализация показала, что современные инструменты позволяют эффективно создавать языки программирования без необходимости ручной реализации всех компонентов.

Разработанный язык Aqua демонстрирует баланс между простотой и выразительностью и может использоваться в образовательных целях для изучения принципов построения языков.

Полученные результаты подтверждают применимость выбранных подходов и инструментов, а также могут служить основой для дальнейших исследований.

Разработка интерпретатора продемонстрировала навыки работы с формальными языками, алгоритмами и архитектурой программных систем, а также способность реализовывать сложные программные решения.

Список использованных источников

Приложения